

第6章. 检索评估、工程验证与项目复盘

怎样证明项目有效，并把学习转化为可信的项目经历？

学习目标

1. 建立人工确认的 Ground Truth 与公平比较方式。
2. 手算 Hit、Recall、MRR，并理解延迟口径。
3. 分层检查纯函数、检索服务、工作流与前端。
4. 用事实、实验和可复现证据完成复盘与简历表达。

阅读方法：先看业务问题和图，再读名词与源码，最后复制完整实验验证。代码节选用于定位实现；完整实验会明确运行条件。

代码基准：2026-10-03 本地项目当前源码。默认配置与环境覆盖值可能不同；本文不以旧实施计划代替真实实现。

运行约定：命令从项目根目录执行。学习实验代码直接收录在 PDF 内；自行新建练习文件后复制运行，不另附脚本。服务实验的结果应由你实际运行记录。

学习产出：每章至少留下一份自己的输入、输出和解释。涉及检索质量、耗时或简历中的改善幅度，只采用有证据的数据。

本章目录

6.1 需求分析：运行成功之后，还要回答什么 3

6.2 Benchmark 与 Ground Truth：先检查当前数据是否可评估 3

6.3 怎样标注：候选辅助与人工判断各自负责什么 5

6.4 四种 Retriever：怎样形成有意义的对照 5

6.5 Hit、Recall 与 MRR：用同一个例子理解三个指标 6

6.6 延迟与百分位：一个平均值为什么不够 8

6.7 Runner 与 Report：指标怎样从单条变成汇总 8

6.8 业务指标：结果数合规与用户任务完成不是一回事 9

6.9 工程验证分层：纯函数、替身、API 与真实服务 10

6.10 完整离线实验：亲手计算一组排名指标 11

6.11 真实服务验证：按场景保留原始结果 12

6.12 失败分析：从错例定位到改进假设 13

6.13 学习复盘：理解、实验与履历怎样连接 13

前面几章说明系统如何运行，本章回答它是否搜得准、交互是否稳定，以及个人学习成果怎样留下可信证据。这里以项目现有评估代码为主，区分真实本地检查、教学数据和需要后续采集的服务结果。PDF 中直接保留实验代码，不提供配套学习脚本。

6.1 需求分析：运行成功之后，还要回答什么

用户输入一句话，系统返回了五条资料，界面也显示正常，这只能证明一次流程得到结果。它还没有证明这五条资料与用户需求相关，相关资料有没有遗漏，或换一种问法之后仍然能找到。

本项目主要输出真实资料条目与分类选项，所以首先评估检索质量和业务流程。飞书课程中还讨论 Faithfulness（忠实性）、Correctness（正确性）等生成质量指标。如果未来加入图纸正文问答，它们会有价值；当前不能把“资料名称返回成功”直接换算成正文回答忠实度。

评估至少需要三类证据：人工确认哪些 ID 与问题相关；程序实际返回哪些 ID、顺序如何；执行耗时和交互过程是否被完整记录。缺少第一类，只能展示搜索结果，无法计算有业务含义的召回率。缺少第三类，就无法判断新增重排是否值得它的时间成本。

阅读入口分别是：eval/metrics.py 中的命中、覆盖与倒数排名函数；eval/runners.py 中的单次检索计时；图 workflow 相关测试中的澄清、改选与恢复规则。它们共同帮助解释结果，但没有任何一个单独的指标能代表全部系统质量。

本节小结：评估先把“正确”的定义写清楚，再选择指标。对于这个项目，“资料条目相关”和“图纸内容解释正确”是不同的判断任务。

6.2 Benchmark 与 Ground Truth：先检查当前数据是否可评估

Benchmark（评估集）是一组固定问题及其标注，用于重复比较不同检索方案。**Ground Truth（标准相关集合）**在本项目中是 expected_ids：人工确认与问题相关的资料 ID 列表。它不是对话模型生成的一段参考答案。

每条 EvalCase 还有 label_status、source 和 notes。只有 label_status == 'verified' 且 expected_ids 非空，is_labeled 才为真。labeled_cases() 依据这个条件筛选样本，正式评估脚本只使用筛选后的结果。

本次真实状态检查：2026-10-03 读取当前 eval/benchmark.jsonl，共 22 条问题，22 条的 expected_ids 全部为空，22 条状态全部为 pending，可评估的已标注样本数量为 0。因此，当前没有依据给出真实的 Hit@5、Recall@5 或 MRR@5 改善幅度。

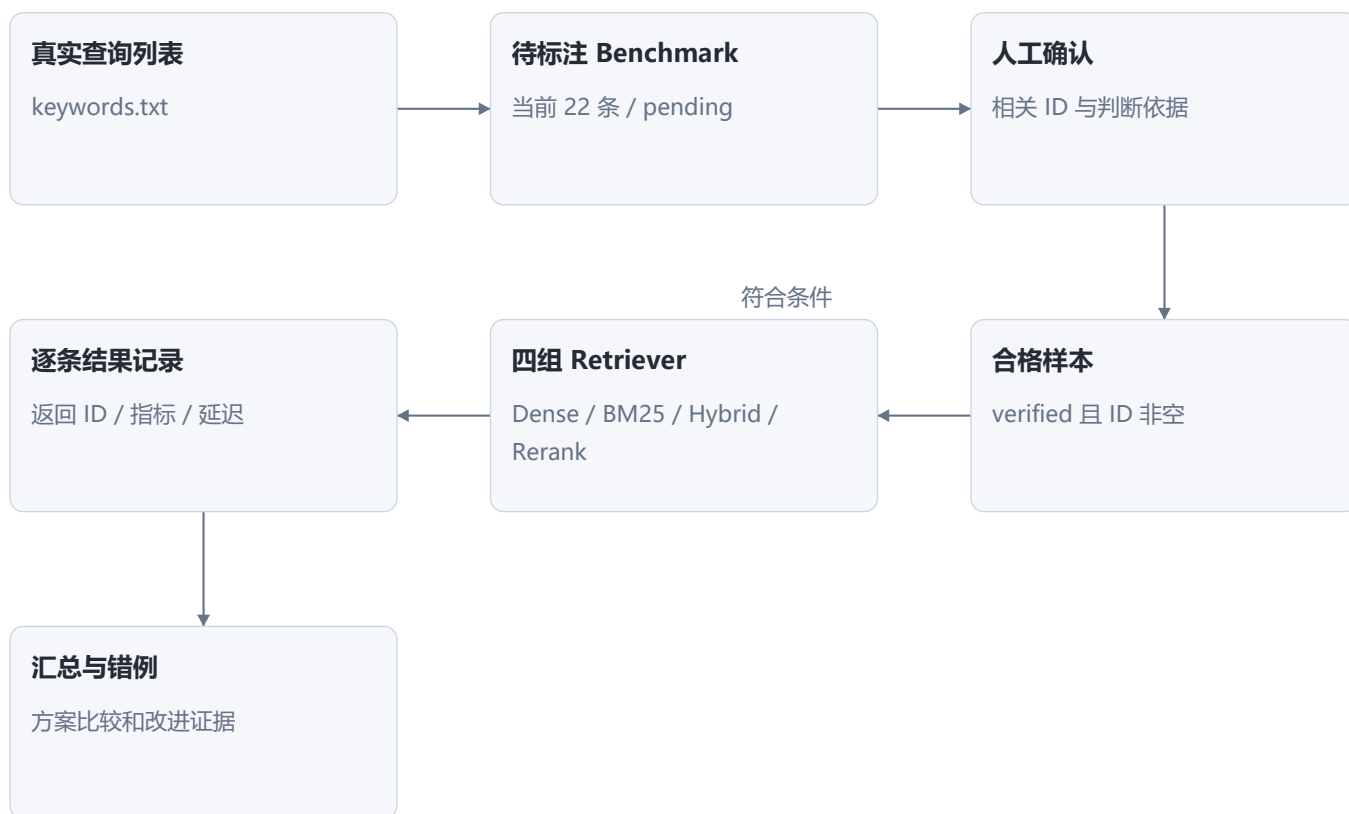


图 6-1. 当前 22 条 Benchmark 全部待标注，须先人工确认相关 ID；只有非空标签且 verified 的样本进入正式排名评估。

源码节选：当前的标注资格判定如下。它是读取源码的片段，不是一个独立实验。

代码块 python

```
1 @property
2 def is_labeled(self) -> bool:
3     return (
4         self.label_status == "verified"
5         and bool(self.expected_ids)
6     )
```

这个规则也带来一个边界：确实不存在相关资料的“无答案问题”无法以空 ID 列表进入当前检索排名评估。它们需要单独的无结果业务用例和误报率统计，或者在扩展评估 Schema 后使用更明确的标签。不能为了让脚本通过，随便填入一个不相关 ID。

易错点：有 22 个查询，不等于有 22 个已验证样本；verified 这个文字本身，也不是人为审核已经完成的证据。应核对具体 ID、资料标题、业务适配条件和备注。

6.3 怎样标注：候选辅助与人工判断各自负责什么

scripts/build_eval_set.py 从 data/keywords.txt 生成问题，默认输出到 eval/benchmark.jsonl。它填入空 expected_ids 和 pending，说明这里只是在建立待标注问题列表。本次没有重新运行这个写入脚本，避免覆盖现有评估文件。

scripts/suggest_eval_labels.py 提供人工标注候选。它先做文本归一化，再计算 Character Bigrams（相邻双字符）的集合重合比例。这个分数用于快速查看可能相关的标题，不是正式 RAG 的召回分数，更不是自动真值。

建议按一个问题完成完整标注流程：理解用户所需品牌、型号和资料类型；查看辅助候选；必要时检查更广的目录；确认所有已知相关 ID；记录同名、型号不清或资料缺失等情况；最后才将状态改为 verified。

完整可运行命令：只读当前目录与评估集，不调用 API，不写入标签。本章只提供入口，本次没有把候选输出当成人工完成的标注。

代码块 powershell

```
1 .venv\Scripts\python.exe -m scripts.suggest_eval_labels --case 1 --top-k 10
```

标注检查项	需要具体确认什么	容易出现的偏差
资料身份	ID 对应真实记录	把排名序号当成 ID
品牌与型号	需求是否适用于该设备	相似标题掩盖型号差异
资料类型	保险丝图、针脚定义等是否相符	主题接近但用途不同
相关集合完整性	是否还有其他相关 ID	只标首个候选导致低估覆盖
不确定条件	能否明确解释歧义	把含糊问题强行标成精确结果

一个重要习惯：不要只看待比较系统的 Top5 来形成标签。这样容易把它已经漏掉的资料排除出标准集合，评估就会偏向这个系统。可以扩大候选池，结合不同检索方式、目录浏览和业务知识，尽量提高标注覆盖。

本节小结：候选工具减少翻目录的工作量，人工判断负责相关性。评估集最好留下版本、标注日期和判断依据，才能解释后续指标变化。

6.4 四种 Retriever：怎样形成有意义的对照

eval/evaluate.py 的 build_retrievers() 构造四组方案：Dense、BM25、Hybrid RRF、Hybrid Rerank。所有方案接收查询字符串和请求结果数量，返回具有 doc_id 的统一结果对象，便于同一套指标函数处理。

Dense 检查语义向量召回；BM25 检查词项召回；Hybrid RRF 合并两路排名；Hybrid Rerank 在融合候选上再调用 Cross-Encoder。比较不是为了证明某个名词更高级，而是观察新增步骤对命中、覆盖、排名和耗时的实际影响。

方案标识	核心路径	适合观察的差异
dense	稠密向量检索	同义描述、语义接近的召回
bm25	Milvus 内置 BM25	型号、缩写和关键字召回
hybrid_rrf	Dense + BM25 + RRF	两路互补是否找回漏项
hybrid_rerank	Hybrid + Cross-Encoder	相关结果能否向前排序

Hybrid 的候选数量至少为配置的 hybrid_candidate_k 和请求 TopK 中的较大者。当前默认候选值是 20，图工作流的默认候选值是 30。二者不是同一次实验里的同一个参数，报告中应说明使用的是哪个入口。

当前四路评估直接传入原始问题字符串，没有执行 Web 图中的意图理解、查询改写、业务硬过滤、澄清与恢复流程。因此，这个评估测的是 Retriever 方案，不是完整 /api/chat 的准确率或响应延迟。

公平比较的条件：保持资料版本、标注集、结果截断值和执行环境一致；记录模型预热、候选数量及过滤配置；分析各类问题而不只看总平均。若一次更换模型同时修改候选数和数据模板，即使分数变好，也很难解释主要原因。

易错点：返回五条并不一定是固定长度；过滤或阈值可能让结果少于五条。实际返回数量应该记录下来，不能把“不足五条”自动补成不存在的资料。

6.5 Hit、Recall 与 MRR：用同一个例子理解三个指标

先设标准相关 ID 为 [2, 4]，系统返回 [1, 2, 3, 8, 9]。其中 ID 2 在第 2 位，ID 4 没出现在前五条。下面所有数字都是这个人为构造的教学案例，不是项目服务实测成绩。

Hit@K (前 K 条是否命中) 只看至少有没有一条相关资料。hit_at_k() 把标准 ID 转为集合，与前 K 条结果求交集：交集非空就是 1，否则就是 0。因而本例 Hit@1 为 0，Hit@5 为 1。

Recall@K (前 K 条召回覆盖) 衡量找回了多少标准相关资料。相关集合有两个 ID，当前找回一个，所以 Recall@5 为 1/2，即 0.5。命中不代表找全，因此 Hit@5 为 1 与 Recall@5 为 0.5 可以同时成立。

MRR@K (截断的平均倒数排名) 在单条问题上先找第一个相关结果的排名，贡献为其倒数。第一条相关资料在第 2 位，本例贡献为 1/2。项目先保存各问题的倒数排名，再在报告里求平均；其他相关资料排名不影响这一项。

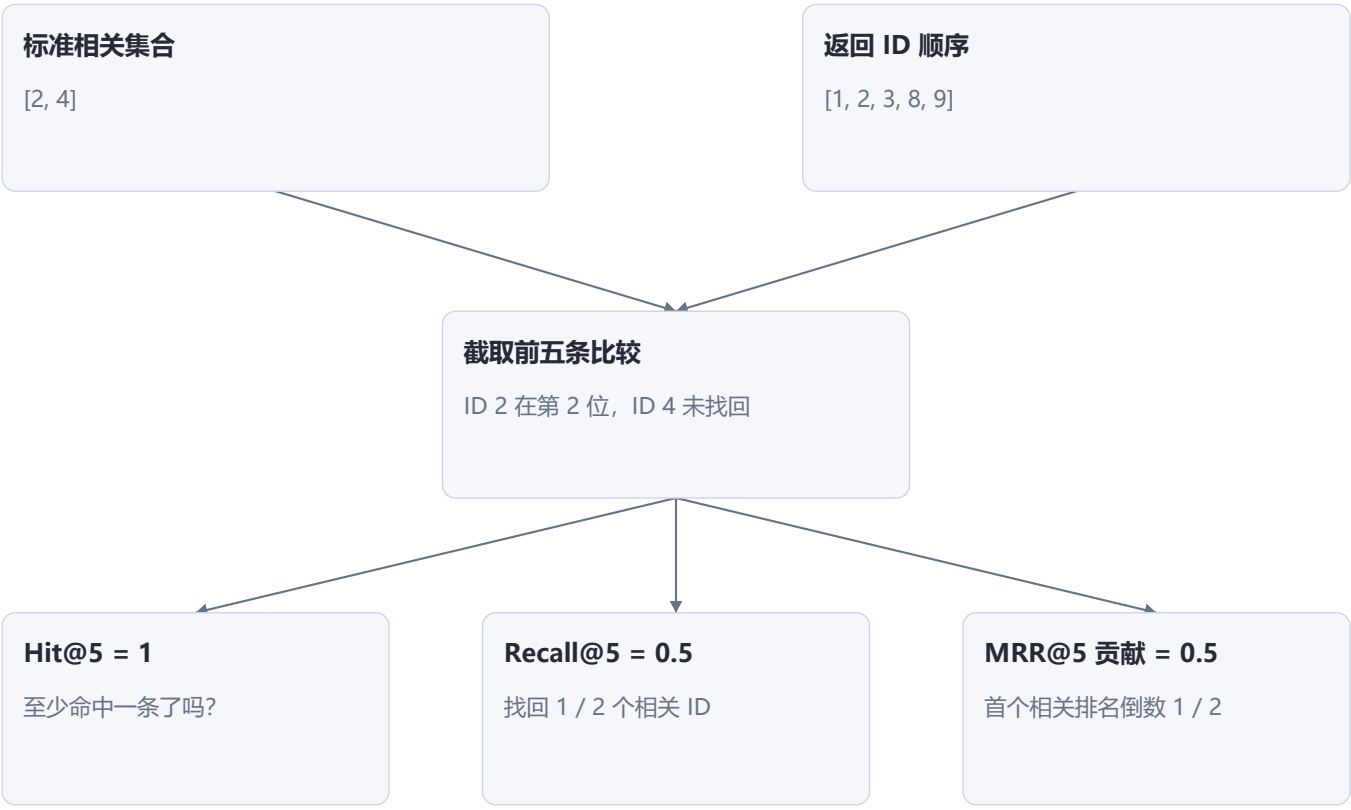


图 6-2. 标准相关 ID=[2,4]，教学返回=[1,2,3,8,9]。命中、覆盖和首个相关排名分别回答不同问题，图中数字不是服务评估成绩。

本例指标	计算依据	数值
Hit@1	第一条 ID 1 不在标准集合	0.0
Hit@5	前五条包含 ID 2	1.0
Recall@5	找回 1 个，共 2 个相关 ID	0.5
MRR@5	第一个相关 ID 排在第 2 位	0.5

如果顺序变成 [2, 1, 3, 8, 9]，Hit@5 和 Recall@5 不变，但 Hit@1 和 MRR@5 提高；如果再把 ID 4 加到前五条，Recall@5 才提高。这能帮助解释重排为何可能提升排名，却不一定提升已截断候选中的覆盖。

源码边界：标准 ID 和 Recall 的返回 ID 都转为集合，所以重复 ID 不会重复增加 Recall；MRR 仍以列表位置计数。若同一 ID 在返回列表中重复出现，既可能浪费展示位置，也可能挤掉其他资料，应另行检查去重。

易错点：Hit 不是 Precision。命中一个相关条目即可得到 Hit=1，即便其他四条都不相关。当前代码未输出 Precision@5，也没有自动计算生成质量指标，学习总结应按已有实现命名。

6.6 延迟与百分位：一个平均值为什么不够

Latency (延迟) 表示操作所花的时间。evaluate_retriever() 用 perf_counter() 在调用 Retriever 前后计时，换算成毫秒。这个范围包含函数调用内发生的模型编码、数据库请求、融合和重排，具体取决于方案。

它不包含完整浏览器交互，也不覆盖 Web 意图模型、图 workflow 恢复等额外步骤。因此报告里的 “P95 ms” 不能直接命名为 “用户端到端响应时间”。如果需要后者，应从客户端发起请求开始，直到可用结果或澄清选项返回，另建采样方法。

Percentile (百分位) 帮助观察分布。P50 对应中间位置，P95 反映较慢一端。本项目把数值排序，使用 $(n-1) * p / 100$ 定位，再对相邻两个值线性插值，不是简单取第 $\text{ceil}(n*p)$ 个样本。

教学数据：假设耗时为 [10, 20, 30, 40, 100] 毫秒。P50 的位置是 2，结果为 30；P95 的位置是 3.8，在 40 与 100 之间插值，结果为 88。88 不一定在原样本中，这符合当前实现。

延迟记录要求	为什么需要	应该怎样写
冷启动与预热	首次模型加载可能影响耗时	单独记录或明确统计范围
CPU / GPU 与设备	编码和重排速度受环境影响	保存设备与模型配置
样本数量	少量样本的尾部估计不稳定	同时报告 Queries
失败与超时	只统计成功项可能低估成本	另列失败数与超时规则
计时边界	Retriever 与端到端不同	在报告名称中说明范围

本次边界：没有运行真实模型服务的延迟对照，所以不填写四种方案的真实 P50/P95 数字。上面的 30 和 88 仅验证指标函数的算法。把教学示例放进性能报告，会让学习材料失去可信度。

6.7 Runner 与 Report：指标怎样从单条变成汇总

Runner 对每条样本调用 Retriever，取返回结果的 doc_id 列表，记录延迟，再计算 Hit@1、Hit@5、Recall@5、MRR@5。Report 按 method 分组，对每条查询的质量指标做等权平均，对耗时计算 P50 和 P95。

等权平均意味着每个问题贡献相同权重。一个有五个标准相关 ID 的问题，不会因为标注更多而在总体 Recall 平均里自动获得更大权重。这与把全部命中数量汇总后再除以全部相关数量的计算方式不同。

源码节选：下面展示当前按查询平均 Recall 的核心计算。

代码块 python

```
1 recall_at_5 = sum(  
2     record.recall_at_5  
3     for record in method_records  
4 ) / count
```

有两个实际边界需要记住。第一，scripts/evaluate_retrieval.py 先调用 labeled_cases()，但 evaluate_retriever() 本身没有再次检查每条样本是否已标注。自行调用函数时，调用者应先筛选有效样本。第二，--top-k 控制向 Retriever 请求的数量，而记录中的指标截断值仍固定为 1 和 5。

如果把 `--top-k` 改成 3，报告仍会列出 Hit@5 等名字，但最多只有三个返回位置可被检查；如果改成 10，质量指标仍只取前五条，计时却包含请求十条带来的工作。这不是对任意 K 都自动改名的评估器，入门时保持默认 5 最容易解释。

需要真实服务后再执行的命令：完成 Ground Truth，并准备好索引和模型后才适用。本次未执行以下命令，也未生成实测结果报告。

代码块 powershell

```
1 .venv\Scripts\python.exe -m scripts.evaluate_retrieval --top-k 5
```

当前没有已标注样本时，这个脚本会明确抛出“没有已验证的 Benchmark”异常。这个失败说明评估前提不足，不应该通过把空标签当成 0 分来绕开。零分应该表示有效样本上没有命中，而不是根本不知道什么是正确资料。

6.8 业务指标：结果数合规与用户任务完成不是一回事

`BusinessEvalSample` 定义了三个字段：最终结果数量、澄清轮数、LLM 调用次数。

`summarize_business_metrics()` 计算最终结果不超过五条的比例、平均澄清轮数、平均模型调用数。

这能描述图工作流的部分行为，但目前只是数据结构和汇总函数。代码不会因为存在这个 Schema 就自动从所有线上会话采集完整统计；还需要明确在哪里记录每轮事件、怎样识别会话完成，以及错误会话如何处理。

当前 `final_le_5_rate` 的判断是 `final_result_count <= 5`，所以数量为 0 也满足条件。如果所有问题都返回 0 条，这项比例仍可能是 100%，但用户没有找到资料。因此，“结果上限合规率”需要和非空成功率、相关性等指标一起看。

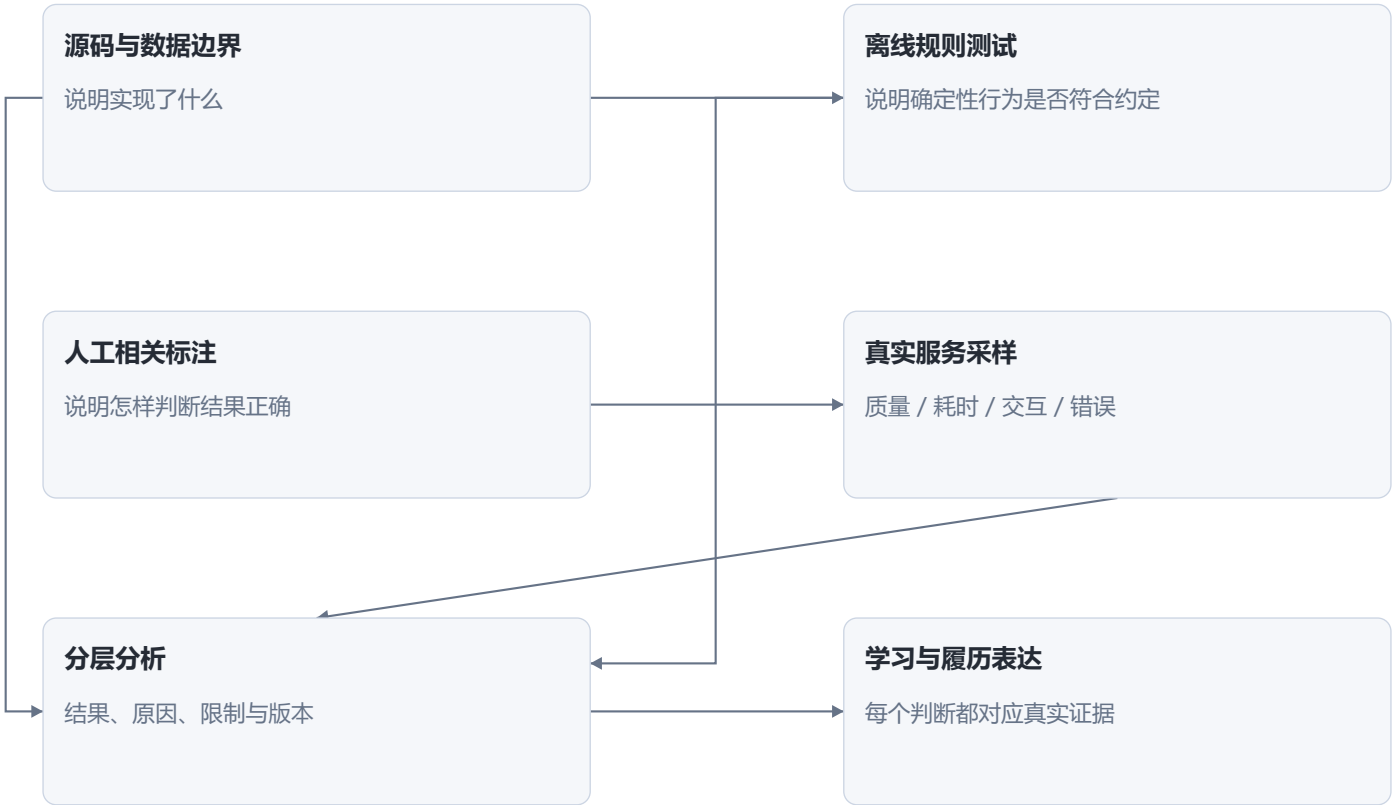


图 6-3. 源码、离线测试、真实标注和服务实测各有范围；把证据汇合后再写项目结论，不能用测试通过替代准确率。

教学解释：两个会话分别返回 0 条和 5 条，都满足数量上限；一个会话澄清三轮后正确收敛，另一个一轮就返回错误资料，前者未必更差。平均轮数是交互成本线索，需要结合成功率和问题难度解释。

建议后续采样：每个会话保存初始问题、候选 ID、澄清维度、选择值、最终 ID、状态、各节点耗时与错误。再区分“找到相关资料”“需要补充条件”“资料库无答案”“系统错误”，这样可以解释用户实际任务是否完成。

易错点：代码中有业务规则，不代表所有真实查询都已经验证；最终不超过五条，也不代表五条都相关。报告里应分别写“规则测试通过”和“人工相关性评估结果”，避免把不同证据混成一个准确率。

6.9 工程验证分层：纯函数、替身、API 与真实服务

Unit Test（单元测试） 验证一个函数或模块的明确行为；**Contract Test（契约测试）** 验证返回字段和请求响应约定；**Smoke Test（冒烟测试）** 快速检查主要流程是否可用。它们的范围不同，结果也应该分层说明。

本次在现有虚拟环境中执行 Loader、向量写入替身、指标函数和 Benchmark 过滤测试，共 12 项通过。Loader 测试使用临时 CSV；向量写入测试通过 FakeVectorStore 替代真实数据库；指标测试使用固定列表。因此这 12 项没有调用模型 API、下载模型或重建索引。

验证层	当前项目例子	能证明与不能证明的范围
纯函数	tests/test_eval_metrics.py	指标计算正确；不证明检索准确
数据转换	tests/test_loader.py	字段与错误处理；不证明数据库同步
替身边界	tests/test_vectorstore_indexing.py	方法选择和主键；不证明真实写入
API / 图契约	tests/test_graph_routes.py 等	分支与结构；依赖替身时不含真服务
真实冒烟	scripts/smoke_test_api.py	真实依赖流程；需记录环境和输出

完整可运行的离线命令：本次实际执行并通过。多行 PowerShell 使用反引号续行，反引号后不要放空格。

代码块 powershell

```
1 .venv\Scripts\python.exe -m pytest `
2     tests/test_loader.py `
3     tests/test_vectorstore_indexing.py `
4     tests/test_eval_metrics.py `
5     tests/test_eval_benchmark.py -q
```

过去文档提过的“59 项测试”属于当时的记录。今天的结果应来自今天执行的命令，不能复制历史数字充当本次验证。测试总数也不代表业务样本规模：12 项测试可以覆盖多个边界，但它仍不是 12 个真实已标注查询。

真实冒烟脚本会访问运行中的 API，检索过程中会触发 LLM 和 Reranker。本章阅读了它的严格通过条件和测试场景，没有运行真实服务测试。脚本注释中的耗时描述是历史说明，不采用为今天的实测性能。

6.10 完整离线实验：亲手计算一组排名指标

下面把第 6.5、6.6 节连起来。只导入项目指标函数，所有输入都是手写教学数据。本次实际执行的数值与下面一致，目的是确认理解计算规则。

完整可运行代码：在项目根 Python 环境执行，无网络或数据库依赖。

代码块 python

```
1 from eval.metrics import (
2     hit_at_k,
3     recall_at_k,
4     mrr_at_k,
5     percentile,
6 )
7
8 expected = [2, 4]
9 retrieved = [1, 2, 3, 8, 9]
10 print("Hit@1:", hit_at_k(expected, retrieved, 1))
11 print("Hit@5:", hit_at_k(expected, retrieved, 5))
12 print("Recall@5:", recall_at_k(expected, retrieved, 5))
13 print("MRR@5:", mrr_at_k(expected, retrieved, 5))
14 demo_latencies = [10, 20, 30, 40, 100]
15 print("教学 P50:", percentile(demo_latencies, 50))
16 print("教学 P95:", percentile(demo_latencies, 95))
```

输出解释：依次得到 0、1、0.5、0.5、30、88。第一组说明没有首位命中，但找回了半数相关资料；第二组说明线性插值能够产生样本中不存在的百分位值。它们都不反映模型真实速度。

练习一：把 ID 2 移到第 1 位，观察哪些指标改变。**练习二：**让 ID 4 也进入前五条，观察 Recall 是否改变。**练习三：**把相关 ID 全部移到第六位之后，观察截断指标为什么变为 0。每次先写自己的预判，再执行验证。

完整可运行的状态检查：只读 Benchmark，帮助判断正式评估是否具备前提。

代码块 python

```
1 from eval.benchmark import load_benchmark, labeled_cases
2
3 cases = load_benchmark("eval/benchmark.jsonl")
4 print("全部问题:", len(cases))
5 print("空标签:", sum(not case.expected_ids for case in cases))
6 print("已标注:", len(labeled_cases(cases)))
```

本次输出为 22、22、0。后续手工标注之后，第三个数字才可能增加；更新学习笔记时应记录新数据版本和标注依据，避免把今天的状态当成永久结论。

6.11 真实服务验证：按场景保留原始结果

项目的冒烟脚本检查健康接口、静态资源、请求校验、普通聊天、检索直答、澄清、改选、返回和过滤回退等行为。验证时不要只看 HTTP 200：业务层可能返回 code=0，或成功结构中只有空列表。

search_response_has_results() 要求 HTTP 200、字典结构、业务 code=1，并且 options 或 documents 非空。测试专门拒绝业务错误与空结果。这说明“接口没有崩溃”与“检索场景通过”是不同检查条件。

场景	要观察什么	应保留的证据
普通问候	走聊天分支，有可读文本	请求与响应类型
精确资料查询	返回真实 ID 与完整字段	标准相关 ID 对照
宽泛查询	给出有意义的澄清分类	选项值、文字与候选数量
同轮或跨轮改选	结果随选择变化	每次选择与返回 ID
返回上一步	恢复此前合理状态	前后选项与候选集合
硬过滤零命中	是否按规则回退检索	过滤表达式和回退标记

实施思路：先建立独立会话验证各场景，再测试同一会话内的状态变化。比较不同选项时，不要让前一个选择污染后一个选择的起点。检查返回上一步时，除了选项文字，也要看对应候选集合是否恢复。

延迟采集建议：对同类问题重复采样，区分第一次初始化与后续请求；同时保存失败或超时。每条记录注明调用的模型、服务版本和机器环境。没有这些信息，换机器后的延迟对比很难解释。

本节边界：这是一份后续实测路线，不是已经通过的服务报告。完成标注与真实服务验证后，才在这一章追加实际数值和典型失败案例。

6.12 失败分析：从错例定位到改进假设

如果某个相关 ID 没有进入候选，首先看 CSV 是否包含它、Loader 是否读取成功、索引是否更新，再分别查看 Dense 和 BM25 的候选。此时问题可能在源数据、模型表达或召回配置，重排无法把根本没召回的资料补出来。

如果相关 ID 进入较大候选集合，却没进最终五条，再看融合排名、Reranker 分数和阈值。若用户指定的型号与资料路径写法不一致，硬过滤可能引起假阴性，还应核对过滤与无过滤回退行为。

如果候选本来正确，但用户选择某个分类后得到另一组条目，优先检查选项值、候选快照、会话 ID 与恢复入口，而不是立即调整 Embedding。故障现象出现在哪一层，决定应该验证哪一层。

一个可复用的错例记录包括原问题、标准 ID、各阶段返回 ID、影响参数、错误类型、预想修复和修复后的验证结果。每次只改变足够少的因素，保留能够解释前后差异的条件。没有错例分析，反复增加候选数很容易让耗时增长，却不知道解决了什么。

易错点：标注不完整也会制造“错误”。系统返回一个实际相关但未列入 `expected_ids` 的条目时，程序会按不相关处理。发现这种情况应先核对并修订标注，而不是为了指标把有效资料从系统结果里排除。

本节小结：优化先提出可验证假设。例如“型号缩写被改写后丢失，导致 BM25 漏召回”，应对应具体错例和改写前后候选。把“提升 RAG 效果”写成这样的因果问题，才方便持续学习。

6.13 学习复盘：理解、实验与履历怎样连接

个人学习可以分成三个成果：说清每个模块负责什么；用离线实验验证关键规则；用真实数据分析一批结果。第一阶段不要急着追求量化提升，先把数据边界、Agent 与 Workflow 的角色、召回与重排的区别讲清楚。

第二阶段完成可重复实验：预览 CSV、检查 ID、用替身验证接口、计算一组指标、观察图工作流的确定性分支。每个实验记录输入、命令、输出和解释。能解释“这次实验没有覆盖什么”，往往比报出测试数量更能体现理解。

第三阶段完成人工标注与真实服务评估，形成包含成功与失败的结果集。此时才有机会写“在哪些查询上改进了什么”，并把证据链接到报告、代码与测试。只有概念介绍，没有执行或证据，不宜写成已经完成的工程贡献。

复盘问题	应能讲清的内容	需要的材料
为什么是 Hybrid?	同义语义与精确编号的互补	分组错例与四路对照
为什么先召回再重排?	覆盖和细粒度排序的分工	候选与最终结果对照
为什么使用 LangGraph?	数量分支、澄清、恢复与改选	状态变化及规则测试
系统现在的限制?	标题目录检索、标签尚未完成	数据检查与当前 Benchmark

可使用的项目介绍句式：“在个人车辆电路图资料检索项目中，梳理 CSV 到 Document 的数据链路，结合 Dense、BM25、RRF 与 Cross-Encoder 形成检索方案，并使用 LangGraph 管理候选数量分支和多轮澄清。”这描述已有架构；如果只是在学习现有代码，应改为“学习并梳理”，不要写成自己独立实现全部功能。

证据完成后再填的履历句式：“在 [数量] 条人工验证问题上，对比 [基线] 与 [改进方案]；[指标] 从 [数值] 变为 [数值]，Retriever P95 为 [毫秒]；针对 [错误类型] 新增 [验证或修复]。”这些空位只能由真实实验填写，目前不能用教学例子的 0.5 或 88 来代替。

面试讲述顺序：先说明用户需求和当前资料边界；再解释数据、检索、Agent 或 Workflow 的分工；选一个具体错例说明怎么定位；最后说明验证方法、结果和限制。这样能让技术名词服务于一个可理解的问题，而不是把框架名字串成清单。

本章小结：当前已经具备可读的评估实现和离线测试基础，但真实标注集仍待完成。后续优先做 Ground Truth、分层实测和错例复盘，再沉淀个人学习记录与可信的项目经历。

延伸阅读：原飞书资料《第3节 RAG评估》。其中检索与生成的分层思路用于理解评估范围；本项目没有引入 RAGAS 自动评估流程，指标定义以 eval/metrics.py 为准，不将课程的工具清单当作项目已实现功能。